

08_tfidf_ventanas_y_shallow

November 28, 2025

0.0.1 08 – TF-IDF de caracteres y modelos shallow para detección de fronteras

En este cuaderno usamos las representaciones TF-IDF de caracteres de la E2 y el dataset de fronteras de la E3 (07) para construir: - Un detector por ventanas deslizantes basado en similitud coseno. - Dos modelos de *shallow learning* (Regresión Logística y SVM lineal) sobre representaciones dispersas.

Todo se ajusta en **train** y se evalúa en **validation**, usando las métricas de segmentación (F1, Pk, WindowDiff).

```
[1]: from pathlib import Path
import json

import numpy as np
import pandas as pd

from scipy import sparse

import matplotlib.pyplot as plt

from sklearn.metrics import (
    accuracy_score,
    f1_score,
    precision_score,
    recall_score,
    classification_report,
)
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.decomposition import TruncatedSVD

RANDOM_SEED = 42
np.random.seed(RANDOM_SEED)
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 120)
plt.style.use("seaborn-v0_8")
```

```
[2]: def find_project_root(marker: str = ".git") -> Path:
    path = Path.cwd()
    for parent in [path, *path.parents]:
        if (parent / marker).exists():
            return parent
    raise RuntimeError(f"No se ha encontrado {marker} en ningún directorio_
↳ padre.")

PROJECT_ROOT = find_project_root()
DATA_PROCESSED_DIR = PROJECT_ROOT / "data" / "processed"
BOUNDARIES_DIR = DATA_PROCESSED_DIR / "boundaries"
FEATURES_DIR = PROJECT_ROOT / "features"
TFIDF_DIR = FEATURES_DIR / "tfidf"
REPORTS_DIR = PROJECT_ROOT / "reports"
REPORTS_DIR.mkdir(exist_ok=True, parents=True)

PROJECT_ROOT, DATA_PROCESSED_DIR, TFIDF_DIR, REPORTS_DIR

[2]: (PosixPath('/Users/eeguskiza/Documents/Deusto/2025/NLP/multi-author-analysis'),
PosixPath('/Users/eeguskiza/Documents/Deusto/2025/NLP/multi-author-
analysis/data/processed'),
PosixPath('/Users/eeguskiza/Documents/Deusto/2025/NLP/multi-author-
analysis/features/tfidf'),
PosixPath('/Users/eeguskiza/Documents/Deusto/2025/NLP/multi-author-
analysis/reports'))

[3]: def boundaries_to_segments(y_boundaries: np.ndarray) -> np.ndarray:
    y_boundaries = np.asarray(y_boundaries, dtype=int)
    n_sentences = len(y_boundaries) + 1
    segments = np.zeros(n_sentences, dtype=int)
    current = 0
    segments[0] = current
    for i, val in enumerate(y_boundaries):
        if val:
            current += 1
            segments[i + 1] = current
    return segments

def pk(reference: np.ndarray, hypothesis: np.ndarray, k: int) -> float:
    reference = np.asarray(reference)
    hypothesis = np.asarray(hypothesis)
    if reference.shape != hypothesis.shape:
        raise ValueError("Las secuencias deben tener la misma longitud")
    if k <= 0 or len(reference) <= k:
        return 0.0
```

```

disagreements = 0
total = 0
for i in range(len(reference) - k):
    same_ref = reference[i] == reference[i + k]
    same_hyp = hypothesis[i] == hypothesis[i + k]
    disagreements += int(same_ref != same_hyp)
    total += 1
return disagreements / total if total else 0.0

def windowdiff(reference: np.ndarray, hypothesis: np.ndarray, k: int) -> float:
    reference = np.asarray(reference, dtype=int)
    hypothesis = np.asarray(hypothesis, dtype=int)
    if reference.shape != hypothesis.shape:
        raise ValueError("Las secuencias deben tener la misma longitud")
    n_sentences = len(reference) + 1
    if k <= 0 or n_sentences <= k:
        return 0.0
    total = n_sentences - k
    errors = 0
    for start in range(total):
        end = start + k - 1
        ref_count = reference[start:end].sum()
        hyp_count = hypothesis[start:end].sum()
        errors += int(ref_count != hyp_count)
    return errors / total if total else 0.0

def evaluate_predictions(df_boundaries: pd.DataFrame, y_pred: np.ndarray, label:
    ↪ str = "model") -> dict:
    df = df_boundaries.sort_values(["level", "split", "doc_id", "boundary_id"]).
    ↪reset_index(drop=True)
    y_true = df["y"].to_numpy().astype(int)
    y_pred = np.asarray(y_pred, dtype=int)
    if len(y_true) != len(y_pred):
        raise ValueError("Vector de predicciones con longitud incorrecta")

    acc = accuracy_score(y_true, y_pred)
    precision = precision_score(y_true, y_pred, average="macro",
    ↪zero_division=0)
    recall = recall_score(y_true, y_pred, average="macro", zero_division=0)
    f1_macro = f1_score(y_true, y_pred, average="macro", zero_division=0)

    doc_records = []
    seg_lengths = []
    for (_, _doc_id), group in df.groupby(["level", "doc_id"]):
        idx = group.index.to_numpy()

```

```

y_doc = group["y"].to_numpy()
doc_records.append((idx, y_doc))
seg = boundaries_to_segments(y_doc)
_, counts = np.unique(seg, return_counts=True)
seg_lengths.extend(counts.tolist())

mean_segment_length = float(np.mean(seg_lengths)) if seg_lengths else 1.0
k = max(1, int(round(mean_segment_length / 2)))

pk_scores = []
wd_scores = []
for idx, y_doc in doc_records:
    y_pred_doc = y_pred[idx]
    seg_true = boundaries_to_segments(y_doc)
    seg_pred = boundaries_to_segments(y_pred_doc)
    pk_scores.append(pk(seg_true, seg_pred, k=k))
    wd_scores.append(windowdiff(y_doc, y_pred_doc, k=k))

metrics = {
    "label": label,
    "accuracy": acc,
    "precision_macro": precision,
    "recall_macro": recall,
    "f1_macro": f1_macro,
    "pk": float(np.mean(pk_scores)) if pk_scores else np.nan,
    "windowdiff": float(np.mean(wd_scores)) if wd_scores else np.nan,
    "k_window": k,
    "mean_segment_length": mean_segment_length,
}
print(f"[{label}] acc={acc:.3f} f1={f1_macro:.3f} pk={metrics['pk']:.3f} _
wd={metrics['windowdiff']:.3f}")
return metrics

```

0.0.2 1. Carga de datos

```

[4]: boundaries_train = pd.read_csv(BOUNDARIES_DIR / "boundaries_train.csv")
boundaries_val = pd.read_csv(BOUNDARIES_DIR / "boundaries_validation.csv")

for name, df in [("train", boundaries_train), ("validation", boundaries_val)]:
    print(f"{name}: {df.shape[0]} fronteras")
    print(df["y"].value_counts(normalize=True).rename("pct_y"))
    print("-" * 40)

```

```

train: 159002 fronteras
y
0    0.809474
1    0.190526
Name: pct_y, dtype: float64

```

```
-----
validation: 33858 fronteras
y
0      0.806013
1      0.193987
Name: pct_y, dtype: float64
-----
```

0.0.3 1.1 Precomputo con scripts externos

Para evitar cuelgues en el notebook se han separado los scripts de entrenamientos, ejecuta en una terminal:

```
python scripts/08_compute_sw.py
```

Ese script calcula las similitudes por ventana, guarda `data/processed/boundaries*_sw.parquet` y las tablas de resultados en `reports/`. Luego vuelve aquí y continúa.

```
[5]: SW_TRAIN_PATH = DATA_PROCESSED_DIR / "boundaries_train_sw.parquet"
SW_VAL_PATH = DATA_PROCESSED_DIR / "boundaries_val_sw.parquet"
RESULTS_SW_PATH = REPORTS_DIR / "08_sw_results.csv"
BEST_SW_JSON = REPORTS_DIR / "08_sw_best.json"

if not SW_TRAIN_PATH.exists() or not SW_VAL_PATH.exists():
    raise FileNotFoundError("Ejecuta scripts/08_compute_sw.py para generar los_
    ↪archivos .parquet")

boundaries_train_sw = pd.read_parquet(SW_TRAIN_PATH)
boundaries_val_sw = pd.read_parquet(SW_VAL_PATH)
results_df = pd.read_csv(RESULTS_SW_PATH)
best = json.loads(BEST_SW_JSON.read_text())

WINDOW_COLS = [c for c in boundaries_train_sw.columns if c.
    ↪startswith("cos_char_w")]
WINDOW_SIZES = sorted(int(c.split("w")[1]) for c in WINDOW_COLS)
print("Columnas disponibles:", WINDOW_COLS)
best
```

```
Columnas disponibles: ['cos_char_w1', 'cos_char_w2', 'cos_char_w3',
'cos_char_w5']
```

```
[5]: {'window': 1.0,
      'tau': 0.7,
      'accuracy': 0.19167054502459088,
      'f1_macro': 0.1618862590772582,
      'pk': 0.6021358093782171,
      'windowdiff': 0.7734709741621023,
      'k_window': 2.0}
```

```
[6]: boundaries_train_sw[WINDOW_COLS].describe()
```

```
[6]:
```

	cos_char_w1	cos_char_w2	cos_char_w3	cos_char_w5
count	159002.000000	159002.000000	159002.000000	159002.000000
mean	0.061380	0.089183	0.108253	0.133419
std	0.089429	0.086519	0.085553	0.085867
min	0.000000	0.000000	0.000000	0.000000
25%	0.007187	0.030397	0.049743	0.073705
50%	0.027176	0.063301	0.086238	0.116127
75%	0.081645	0.121535	0.144148	0.173256
max	1.000000	1.000000	1.000000	1.000000

```
[7]: for col in WINDOW_COLS:
      w = col.split('w')[1]
      fig, ax = plt.subplots(figsize=(6,4))
      ax.hist(boundaries_train_sw[col], bins=50, alpha=0.7, color='steelblue')
      ax.set_title(f"Distribución de cosenos - window {w}")
      ax.set_xlabel('Similitud coseno')
      ax.set_ylabel('Frecuencia')
      out_path = REPORTS_DIR / f"08_cosine_hist_w{w}.png"
      plt.tight_layout()
      fig.savefig(out_path, dpi=150)
      plt.close(fig)
      print(f"Histograma guardado en {out_path}")
```

Histograma guardado en /Users/eeguskiza/Documents/Deusto/2025/NLP/multi-author-analysis/reports/08_cosine_hist_w1.png

Histograma guardado en /Users/eeguskiza/Documents/Deusto/2025/NLP/multi-author-analysis/reports/08_cosine_hist_w2.png

Histograma guardado en /Users/eeguskiza/Documents/Deusto/2025/NLP/multi-author-analysis/reports/08_cosine_hist_w3.png

Histograma guardado en /Users/eeguskiza/Documents/Deusto/2025/NLP/multi-author-analysis/reports/08_cosine_hist_w5.png

0.0.4 2. Búsqueda de umbral para el modelo de ventanas

```
[8]: def predict_from_cos(df: pd.DataFrame, col: str, tau: float) -> np.ndarray:
      return (df[col].to_numpy() < tau).astype(int)
```

```
TAU_GRID = np.linspace(0.70, 0.98, 15)
results = []
```

```
for w in WINDOW_SIZES:
    col = f"cos_char_w{w}"
    for tau in TAU_GRID:
        y_pred = predict_from_cos(boundaries_train_sw, col, tau)
```

```

        metrics = evaluate_predictions(boundaries_train_sw, y_pred,
        ↪label=f"sw_char_w{w}_tau{tau:.3f}")
        metrics["window"] = w
        metrics["tau"] = tau
        results.append(metrics)

results_df = pd.DataFrame(results)
results_df.sort_values(["f1_macro", "pk", "windowdiff"], ascending=[False,
        ↪True, True]).head()

```

```

[sw_char_w1_tau0.700] acc=0.192 f1=0.162 pk=0.602 wd=0.773
[sw_char_w1_tau0.720] acc=0.192 f1=0.162 pk=0.602 wd=0.774
[sw_char_w1_tau0.740] acc=0.192 f1=0.162 pk=0.602 wd=0.774
[sw_char_w1_tau0.760] acc=0.191 f1=0.162 pk=0.602 wd=0.774
[sw_char_w1_tau0.780] acc=0.191 f1=0.162 pk=0.602 wd=0.774
[sw_char_w1_tau0.800] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w1_tau0.820] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w1_tau0.840] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w1_tau0.860] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w1_tau0.880] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w1_tau0.900] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w1_tau0.920] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w1_tau0.940] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w1_tau0.960] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w1_tau0.980] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w2_tau0.700] acc=0.192 f1=0.162 pk=0.602 wd=0.774
[sw_char_w2_tau0.720] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w2_tau0.740] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w2_tau0.760] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w2_tau0.780] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w2_tau0.800] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w2_tau0.820] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w2_tau0.840] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w2_tau0.860] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w2_tau0.880] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w2_tau0.900] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w2_tau0.920] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w2_tau0.940] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w2_tau0.960] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w2_tau0.980] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w3_tau0.700] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w3_tau0.720] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w3_tau0.740] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w3_tau0.760] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w3_tau0.780] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w3_tau0.800] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w3_tau0.820] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w3_tau0.840] acc=0.191 f1=0.161 pk=0.602 wd=0.774

```

```

[sw_char_w3_tau0.860] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w3_tau0.880] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w3_tau0.900] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w3_tau0.920] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w3_tau0.940] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w3_tau0.960] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w3_tau0.980] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w5_tau0.700] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w5_tau0.720] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w5_tau0.740] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w5_tau0.760] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w5_tau0.780] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w5_tau0.800] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w5_tau0.820] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w5_tau0.840] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w5_tau0.860] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w5_tau0.880] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w5_tau0.900] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w5_tau0.920] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w5_tau0.940] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w5_tau0.960] acc=0.191 f1=0.161 pk=0.602 wd=0.774
[sw_char_w5_tau0.980] acc=0.191 f1=0.161 pk=0.602 wd=0.774

```

```

[8]:          label accuracy precision_macro recall_macro f1_macro
pk windowdiff k_window \
0  sw_char_w1_tau0.700 0.191671          0.487425      0.499836 0.161886
0.602136    0.773471          2
1  sw_char_w1_tau0.720 0.191601          0.486326      0.499831 0.161779
0.602143    0.773512          2
2  sw_char_w1_tau0.740 0.191538          0.483549      0.499805 0.161690
0.602143    0.773556          2
3  sw_char_w1_tau0.760 0.191494          0.485749      0.499841 0.161609
0.602138    0.773585          2
4  sw_char_w1_tau0.780 0.191450          0.484972      0.499839 0.161540
0.602110    0.773620          2

      mean_segment_length window tau
0                4.000606      1 0.70
1                4.000606      1 0.72
2                4.000606      1 0.74
3                4.000606      1 0.76
4                4.000606      1 0.78

```

```

[9]: best = results_df.sort_values(["f1_macro", "pk", "windowdiff"],
    ↪ascending=[False, True, True]).iloc[0]
best

```

```
[9]: label          sw_char_w1_tau0.700
    accuracy          0.191671
    precision_macro    0.487425
    recall_macro       0.499836
    f1_macro           0.161886
    pk                 0.602136
    windowdiff         0.773471
    k_window           2
    mean_segment_length 4.000606
    window             1
    tau                0.7
    Name: 0, dtype: object
```

```
[10]: fig, ax = plt.subplots(figsize=(7, 4))
    for w in WINDOW_SIZES:
        col = f"cos_char_w{w}"
        subset = results_df[results_df["window"] == w]
        ax.plot(subset["tau"], subset["f1_macro"], marker="o", label=f"w={w}")
    ax.set_xlabel("Tau")
    ax.set_ylabel("F1 macro (train)")
    ax.set_title("Modelo de ventanas: F1 vs tau")
    ax.legend()
    plt.tight_layout()
    out_path = REPORTS_DIR / "08_sw_f1_vs_tau.png"
    fig.savefig(out_path, dpi=150)
    plt.close(fig)
    print(f"Figura guardada en {out_path}")
```

Figura guardada en /Users/eeguskiza/Documents/Deusto/2025/NLP/multi-author-analysis/reports/08_sw_f1_vs_tau.png

0.0.5 3. Evaluación del mejor modelo de ventanas

```
[11]: BEST_W = int(best["window"])
    BEST_TAU = float(best["tau"])
    BEST_COL = f"cos_char_w{BEST_W}"

    print(f"Mejor ventana: {BEST_W} | tau: {BEST_TAU:.3f}")

    y_pred_train_best = predict_from_cos(boundaries_train_sw, BEST_COL, BEST_TAU)
    y_pred_val_best = predict_from_cos(boundaries_val_sw, BEST_COL, BEST_TAU)

    metrics_train_best = evaluate_predictions(boundaries_train_sw,
        ↪ y_pred_train_best, label="sw_char_best_train")
    metrics_val_best = evaluate_predictions(boundaries_val_sw, y_pred_val_best,
        ↪ label="sw_char_best_val")
```

```

p_change_train = boundaries_train_sw["y"].mean()
y_pred_val_never = np.zeros(len(boundaries_val_sw), dtype=int)
y_pred_val_random = np.random.default_rng(RANDOM_SEED).binomial(1,
    ↪p_change_train, size=len(boundaries_val_sw))
metrics_val_never = evaluate_predictions(boundaries_val_sw, y_pred_val_never,
    ↪label="never_change_val")
metrics_val_random = evaluate_predictions(boundaries_val_sw, y_pred_val_random,
    ↪label="random_p_train_val")

comparison = pd.DataFrame([
    metrics_val_never,
    metrics_val_random,
    metrics_val_best,
])
comparison = comparison[["label", "accuracy", "f1_macro", "pk", "windowdiff"]]
comparison

```

Mejor ventana: 1 | tau: 0.700

[sw_char_best_train] acc=0.192 f1=0.162 pk=0.602 wd=0.773

[sw_char_best_val] acc=0.197 f1=0.166 pk=0.600 wd=0.772

[never_change_val] acc=0.806 f1=0.446 pk=0.400 wd=0.228

[random_p_train_val] acc=0.687 f1=0.495 pk=0.476 wd=0.335

```

[11]:
      label  accuracy  f1_macro      pk  windowdiff
0  never_change_val  0.806013  0.446294  0.400211    0.227808
1  random_p_train_val  0.687046  0.494923  0.475761    0.335280
2  sw_char_best_val   0.196556  0.166271  0.599574    0.772016

```

```

[12]: comparison.to_csv(REPORTS_DIR / "08_resumen_sw.csv", index=False)
print("Resumen guardado en", REPORTS_DIR / "08_resumen_sw.csv")

```

Resumen guardado en /Users/eeguskiza/Documents/Deusto/2025/NLP/multi-author-analysis/reports/08_resumen_sw.csv

0.0.6 4. Modelos shallow con TF-IDF

python scripts/08_train_shallow.py --components 128

Esto generará data/processed/X*_delta.npz, y*_delta.npy y los reportes reports/08_logreg_metrics.json y reports/08_svm_metrics.json.

```

[13]: LOGREG_JSON = REPORTS_DIR / "08_logreg_metrics.json"
SVM_JSON = REPORTS_DIR / "08_svm_metrics.json"

if not LOGREG_JSON.exists() or not SVM_JSON.exists():
    raise FileNotFoundError("Ejecuta scripts/08_train_shallow.py para generar
    ↪las métricas")

logreg_info = json.loads(LOGREG_JSON.read_text())

```

```

svm_info = json.loads(SVM_JSON.read_text())

metrics_val_logreg = logreg_info["metrics_val"]
metrics_val_logreg["label"] = logreg_info.get("label", "logreg_tfidf")
metrics_val_svm = svm_info["metrics_val"]
metrics_val_svm["label"] = svm_info.get("label", "svm_tfidf")

print("LogReg - validation")
print(logreg_info["classification_report"])
print("LinearSVM - validation")
print(svm_info["classification_report"])

```

LogReg - validation

	precision	recall	f1-score	support
0	0.823	0.820	0.821	27290
1	0.262	0.266	0.264	6568
accuracy			0.713	33858
macro avg	0.543	0.543	0.543	33858
weighted avg	0.714	0.713	0.713	33858

LinearSVM - validation

	precision	recall	f1-score	support
0	0.822	0.859	0.840	27290
1	0.278	0.226	0.249	6568
accuracy			0.736	33858
macro avg	0.550	0.542	0.545	33858
weighted avg	0.716	0.736	0.725	33858

0.0.7 5. Tabla resumen y comentarios finales

```

[14]: summary_rows = [
    {k: v for k, v in metrics_val_never.items()},
    {k: v for k, v in metrics_val_random.items()},
    {k: v for k, v in metrics_val_best.items()},
    {k: v for k, v in metrics_val_logreg.items()},
    {k: v for k, v in metrics_val_svm.items()},
]
summary_df = pd.DataFrame(summary_rows)
summary_df = summary_df[["label", "accuracy", "f1_macro", "pk", "windowdiff"]]
summary_df = summary_df.sort_values("f1_macro", ascending=False)
summary_df.to_csv(REPORTS_DIR / "08_resumen_modelos.csv", index=False)
summary_df

```

```

[14]:          label  accuracy  f1_macro      pk  windowdiff
4      svm_tfidf  0.735897  0.544605  0.432065    0.281982
3      logreg_tfidf  0.712535  0.542819  0.449466    0.302381
1  random_p_train_val  0.687046  0.494923  0.475761    0.335280
0      never_change_val  0.806013  0.446294  0.400211    0.227808
2      sw_char_best_val  0.196556  0.166271  0.599574    0.772016

```